Programming Fundamentals *LAB* – Spring 2026

(BS-IT-F25 Morning & Afternoon)

LAB # 6

Submission Deadline: ***Saturday, March 14, 2026 (10:00 PM)***

Instructions

- **This is an individual lab. Absolutely NO collaboration is allowed. Any traces of plagiarism/cheating or usage of AI tools will result in an "F" grade in this course and lab.**
- Do **NOT** copy even a single line of code from any other person or book or Internet or any other source.
- This assignment needs to be submitted in **Soft** form. The **Submission Procedure** is given below.
- Late submissions will NOT be accepted, whatever your excuse may be.

Submission Procedure

i. Make 6 .CPP files for the 6 tasks. Name each file in the following format:

```
TASK1.cpp
TASK2.cpp
...
TASK10.cpp
```

Include your **Name, Roll Number, and Section** in comments at the top of each CPP file.

ii. Create an empty folder. Place only the .CPP files inside. **Do NOT include any other files — your submission will NOT be graded otherwise.**

iii. Compress the folder as a **.ZIP** file. Name it exactly as:

```
AI-l6-BAIF25M123   (Morning section)
IT-l6-BITF25A456   (Afternoon section)

Replace the highlighted part with your complete Roll Number.
```

iv. Carefully check your .ZIP file to ensure all instructions above have been followed.

v. Submit the single .ZIP file on **Google Classroom**. Make sure you press the **SUBMIT** button after uploading.

Students Without Laptops:

- Write your Name, Roll Number, and Section at the top of each page.
- Scan your handwritten lab in PDF format using CamScanner, Adobe Scan, or similar app.
- Rename the PDF using the same format provided above.
- Submit your PDF file through Google Classroom.

⚠ **Note:** If you do not follow the above submission procedure, your LAB will NOT be graded and you will be awarded ZERO marks.

TASK # 1 *Custom Multiplication Table (Matrix)*

Write a program that generates a customized multiplication table (matrix) based on a range provided by the user. Instead of starting from 1, the table starts and ends at specific integers, creating a perfect square grid of products.

Requirements

- **User Input:** Ask for a Start Value and an End Value (e.g., 5 and 10).
- **Nested Loop Logic:** Outer loop controls rows; inner loop controls columns — both iterate from Start to End.
- **Formatting:** First row is a header showing column numbers. First column shows the row multiplier. Use setw() for alignment.

Sample Output

```
Enter the starting number: 5
Enter the ending number: 8

     5    6    7    8
  --------------------
5 |  25   30   35   40
6 |  30   36   42   48
7 |  35   42   49   56
8 |  40   48   56   64
```

TASK # 2 *Chessboard Pattern Generator*

Write a program that generates a grid of characters ('X' and 'O') that alternates both horizontally and vertically — resembling a traditional chessboard where no two identical characters are adjacent.

Requirements

- **User Input:** Ask for the grid size (e.g., 8 for an 8×8 grid).

Sample Output (Size 5)

```
Enter the size of the board: 5

X O X O X
O X O X O
X O X O X
O X O X O
X O X O X
```

TASK # 3 *Factor Finder*

A factor of a number is any integer that divides it evenly with no remainder. For example, 6 can be divided evenly by 1, 2, 3, and 6. Finding factors is fundamental in number theory — used in cryptography, simplifying fractions, and divisibility problems.

Write a C++ program that asks the user for an upper limit N, then for every number from 1 to N, finds and displays all of its factors.

Constraints

- N must be a positive integer greater than 0.
- Every number has at least 2 factors (1 and itself), except 1 which has only one factor.

Sample Output (N = 10)

```
Enter upper limit N: 10

Factors of 1  : 1
Factors of 2  : 1 2
Factors of 3  : 1 3
Factors of 4  : 1 2 4
Factors of 5  : 1 5
Factors of 6  : 1 2 3 6
Factors of 7  : 1 7
Factors of 8  : 1 2 4 8
Factors of 9  : 1 3 9
Factors of 10 : 1 2 5 10
```

TASK # 4 *Fibonacci Triangle*

The **Fibonacci sequence** is a series of numbers where each number (starting from the third) is the sum of the two preceding ones. **The Sequence:** 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55……

Write a program that asks for a number of rows and prints a triangle where each row contains the Fibonacci sequence up to that point.

Requirements

- **User Input:** Number of rows (e.g., 5).

Sample Output (Size 5)

```
0
0 1
0 1 1
0 1 1 2
0 1 1 2 3
```

TASK # 5 *Number Conversion System*

Develop a menu-driven system that allows a user to input a number in one format and convert it into another.

Conversion Modes

1. Decimal to Binary (The "Remainder" Method)

- Ask the user for a positive integer (Base 10).
- Use a while loop to repeatedly divide by 2 and store the remainders (0 or 1).

- Challenge: remainders are generated in reverse order — display them from last to first.

2. Binary to Decimal (The "Power" Method)

- Ask the user a number in binary.
- Use a for loop to iterate through each digit, multiplying by 2^n where n is the position from the right.

3. Decimal to Hexadecimal (The "Alpha-Numeric" Map)

- Similar to the binary method but divide by 16.
- If remainder is 10–15, output the letters A through F (10=A, 11=B, ..., 15=F).

Sample Output

```
--- NUMBER CONVERSION SYSTEM ---
1. Decimal to Binary
2. Binary to Decimal
3. Decimal to Hexadecimal
4. Exit

Select an option: 1
Enter Decimal Number: 25
Binary Equivalent: 11001

Would you like to continue? (Y/N)
```

TASK # 6 *Armstrong Number Scanner*

An **Armstrong number** (also known as a **Narcissistic number**) is a number that is equal to the sum of its own digits, each raised to the power of the number of digits in that number.

Write a program that scans all integers from 1 up to a limit N entered by the user. Identify and print every Armstrong Number found in that range.

Requirements

- **User Input:** An integer representing the upper limit of the search (e.g., 1000).
- **The Inspector —** For each number i, perform two sub-tasks:
  - Task A: Count how many digits are in i.
  - Task B: Calculate the sum of each digit raised to the power of that count.
- **The Verification:** If the calculated sum equals i, display it as an Armstrong number.

Sample Output (N = 500)

```
Enter upper limit N: 500
Scanning from 1 to 500...
--------------------------------------
[FOUND]  1   (1-digit)
[FOUND]  2   (1-digit)
...
[FOUND]  9   (1-digit)
[FOUND]  153 (3-digit)
[FOUND]  370 (3-digit)
[FOUND]  371 (3-digit)
[FOUND]  407 (3-digit)
```

```
-------------------------------------
Total Armstrong Numbers Found: 13
```

GOOD LUCK!